

<https://salsa.debian.org/xorg-team/lib/mesa/-/blob/debian-trixie-backports/debian/rules>

This is a very well-structured `debian/rules` file. You are exactly right: Debian's current packaging handles the `d3d12` Gallium driver (which provides OpenGL and OpenCL over D3D12), but it does **not** build the `microsoft-experimental` (also known as `dzn`) driver, which provides Vulkan over D3D12.

If you just append `microsoft-experimental` to the Vulkan drivers list, `dh_install` will fail. Debian's packaging rules (`debian/*.install` files) don't know what to do with the resulting `vulkan_dzn.so` and `vulkan_dzn.json` files, because they aren't explicitly declared in the official packaging.

To answer your question directly: **You cannot easily build "just the missing parts" as a standalone side-package while using the rest of Debian's Mesa.**

Mesa's build system (Meson) is deeply integrated. Drivers share core libraries (like `libgallium`), and you cannot instruct Meson to compile only a Vulkan driver without also compiling the core Mesa infrastructure it links against. If you tried to build just the missing `dzn` driver and drop it into a system running the official `trixie-backports` Mesa, you would likely encounter symbol mismatches or segmentation faults due to mismatched `libgallium` versions.

If you want Vulkan via `dzn` on WSL2 natively integrated into Debian, you must rebuild the entire Debian Mesa source package and add a custom package entry to house the `dzn` files.

How to add `vulkan-dzn` to the Debian Build

To do this successfully, you need to modify three separate areas of the Debian source tree.

1. Update the Build Flags (`debian/rules`)

You need to tell Meson to compile the driver for WSL architectures (amd64 and arm64).

Find this section in `debian/rules` :

Makefile

```
# WSL supports only amd64 and arm64
ifneq (, $(filter $(DEB_HOST_ARCH), amd64 arm64))
    GALLIUM_DRIVERS += d3d12
endif
```

Modify it to include the Vulkan driver:

Makefile

```
# WSL supports only amd64 and arm64
ifneq (, $(filter $(DEB_HOST_ARCH), amd64 arm64))
    GALLIUM_DRIVERS += d3d12
    VULKAN_DRIVERS += microsoft-experimental
endif
```

2. Create the Package Definition (`debian/control.in`)

Before the build system can package the resulting files, it needs a deb package definition to hold them. You must add a new paragraph to the end of `debian/control.in` (do not edit `debian/control` directly, as the `rules` file regenerates it from the `.in` template).

Open `debian/control.in` and append this to the very bottom:

Plaintext

```
Package: mesa-wsl-vulkan-dzn
Architecture: amd64 arm64
Depends:
    ${shlibs:Depends},
    ${misc:Depends},
Description: Mesa Vulkan drivers for WSL2 (microsoft-experimental)
    This package provides the Dozen (dzn) Vulkan driver, allowing Vulkan
    applications to run on Windows Subsystem for Linux (WSL2) via D3D12.
```

3. Tell Debian Where to Put the Files (`debian/mesa-wsl-vulkan-dzn.install`)

Finally, the build system needs to know which compiled files belong in this new package. You create a new `.install` file matching the package name you just defined.

Create a new text file named `debian/mesa-wsl-vulkan-dzn.install.in` and add the following lines:

Plaintext

```
usr/lib/${DEB_HOST_MULTIARCH}/libvulkan_dzn.so  
usr/share/vulkan/icd.d/dzn_icd.*.json
```

The Build Process

Once you have made those three changes:

Run `debian/rules regen_control` to generate the updated `debian/control` file.

Run `dch --local +wsl "Added microsoft-experimental driver in custom wsl package"` to bump the version.

Run `dpkg-buildpackage -b -uc -us` to compile everything.

You will now get all the standard Debian Mesa packages, **plus** a new `mesa-wsl-vulkan-dzn_*.deb` file. You can install all of them using `sudo dpkg -i *.deb`, and `apt` will gladly manage them while keeping your KDE 'plow' environment fully accelerated for both OpenGL (via `d3d12` in `libgl1-mesa-dri`) and Vulkan (via `dzn` in your new package).